

Lab Report: Week 6 Session B Lab 4

Flood Inundation Analysis

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 6 Session B Lab 4: **synthetic DEM** generation, **flat-water inundation** mapping, depth statistics, and a **rising-stage** sensitivity over flood levels 40–60 m. A 100×100 grid with 1 m cells and random seed 42 yields elevations 35.96–68.81 m. Wet cells use `elevation <= flood_level`; depth is `max(0, flood_level - elevation)` on wet cells only. At 40, 50, and 60 m, flooded area is 2.41%, 25.25%, and 82.93% with mean wet depth 1.18, 4.86, and 8.33 m. Exercise 4 swept levels in 1 m steps: **Monotonicity check: PASS**. Evidence: one combined terminal screenshot (Figure 1), three figures, and code in `week6_session_b_lab4/` (Appendix A–E). OpenCode was not required.

1 Introduction

Flood risk screening often starts from a digital elevation model (DEM) and a assumed water surface elevation. This lab practices GIS-style inundation on a **synthetic terrain**: generate DEM data, apply a simple depth rule, visualize extent, and verify that flooded area grows monotonically as stage rises—after reservoir optimization (Week 6 Session A Lab 3) and hydrologic modeling (Weeks 3–5).

2 Objectives

- Process (synthetic) DEM data for spatial analysis (Exercise 1).
- Implement `simulate_flood` and area/depth statistics (Exercise 2).
- Create publication-style inundation maps (Exercise 3).
- Simulate rising water levels and check monotonic flooded area (Exercise 4).

3 Methods

3.1 Flood level definition

Following the lab guide, `flood_level` is **absolute stage** (m) in the same vertical datum as the DEM—not depth above a channel thalweg unless redefined.

3.2 Inundation rules

- **Wet mask:** `dem <= flood_level` (inclusive at the water surface).
- **Depth:** `max(0, flood_level - dem)` on wet cells; 0 elsewhere.
- **Grid:** 100×100 cells, **1 m** spacing; seed 42 for reproducibility.

Table 1: Submitted files in week6_session_b_lab4/.

File	Role
flood_inundation.py	Core simulate_flood, stats, monotonicity check
generate_dem.py	Exercise 1: DEM + heatmap
run_flood_tests.py	Exercise 2: levels 40, 50, 60 m
visualize_floods.py	Exercise 3: side-by-side maps (300 DPI)
rising_water.py	Exercise 4: 40–60 m sweep + plot
data/dem.npy, figures/*.png	Saved outputs
prompt_log.md	AI interaction log

3.3 Scripts

4 Results

4.1 Terminal verification

Figure 1 shows one screenshot with all four commands (`generate_dem.py`, `run_flood_tests.py`, `visualize_floods.py`, `rising_water.py`), matching the lab guide’s optional single-screenshot submission style.

```

mahmudul-robotics@robotics-at:~/Downloads/Software Development/ai_water_lab/week6_session_b_lab4$ python3 generate_dem.py
Saved DEM: data/dem.npy shape=(100, 100)
Elevation range: 35.96 - 68.81 m | cell size: 1.0 m | seed: 42
Saved figures/dem_heatmap.png
mahmudul-robotics@robotics-at:~/Downloads/Software Development/ai_water_lab/week6_session_b_lab4$ python3 run_flood_tests.py
Flood_level (m) | Flooded % | Mean depth (m) | Wet cells
-----
40.0 | 2.41 | 1.183 | 241
50.0 | 25.25 | 4.863 | 2525
60.0 | 82.93 | 8.329 | 8293
Convention: wet if elevation <= flood_level; depth=0 off wet cells.
mahmudul-robotics@robotics-at:~/Downloads/Software Development/ai_water_lab/week6_session_b_lab4$ python3 visualize_floods.py
visualize_floods.py:53: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.
  fig.tight_layout()
Saved figures/flood_comparison_300dpi.png
mahmudul-robotics@robotics-at:~/Downloads/Software Development/ai_water_lab/week6_session_b_lab4$ python3 rising_water.py
Levels 40.0-60.0 m step 1.0 m
Flooded % at 40 m: 2.41 | at 60 m: 82.93
Monotonicity check: PASS
Saved figures/flooded_pct_vs_level.png
mahmudul-robotics@robotics-at:~/Downloads/Software Development/ai_water_lab/week6_session_b_lab4$

```

Figure 1: Terminal evidence: DEM saved (35.96–68.81 m), flood table at 40/50/60 m, comparison figure saved, monotonicity **PASS**.

4.2 Exercise 2: flood levels

Table 2: Flooded area and mean depth on wet cells (from `run_flood_tests.py`).

Flood level (m)	Flooded (%)	Mean depth (m)	Wet cells
40	2.41	1.183	241
50	25.25	4.863	2525
60	82.93	8.329	8293

Flooded fraction increases sharply between 50 and 60 m because most valley cells lie below 60 m but many ridges remain dry until stage approaches the DEM maximum.

4.3 Exercise 1: DEM

If `week6_session_b_lab4_dem.png` is missing on Overleaf, use `week6_session_b_lab4_files/figures/dem_hea`

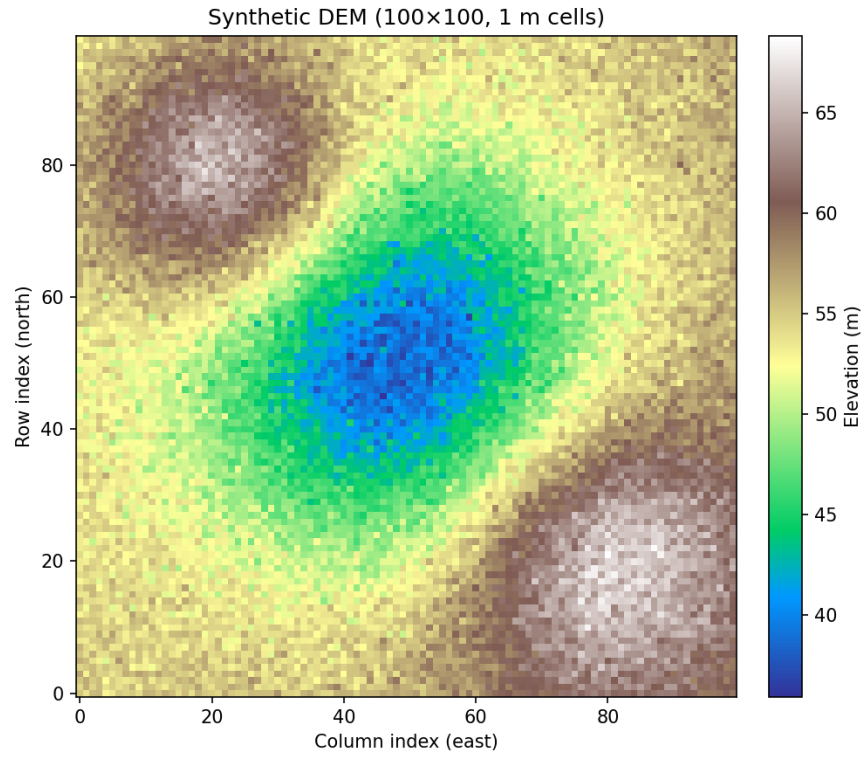


Figure 2: Synthetic DEM heatmap (figures/dem_heatmap.png).

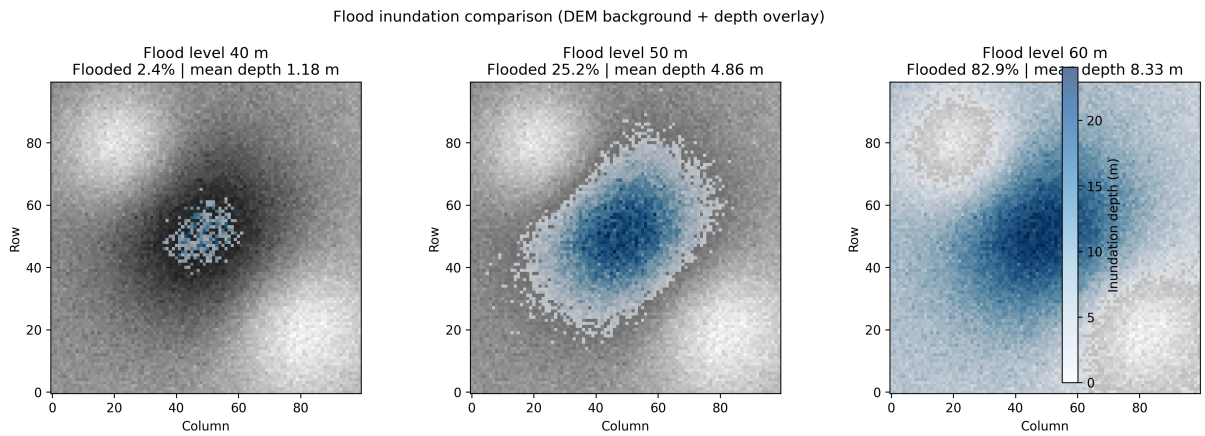


Figure 3: Side-by-side inundation at 40, 50, and 60 m (grayscale DEM, blue depth overlay).

4.4 Exercise 3: inundation maps

4.5 Exercise 4: rising water

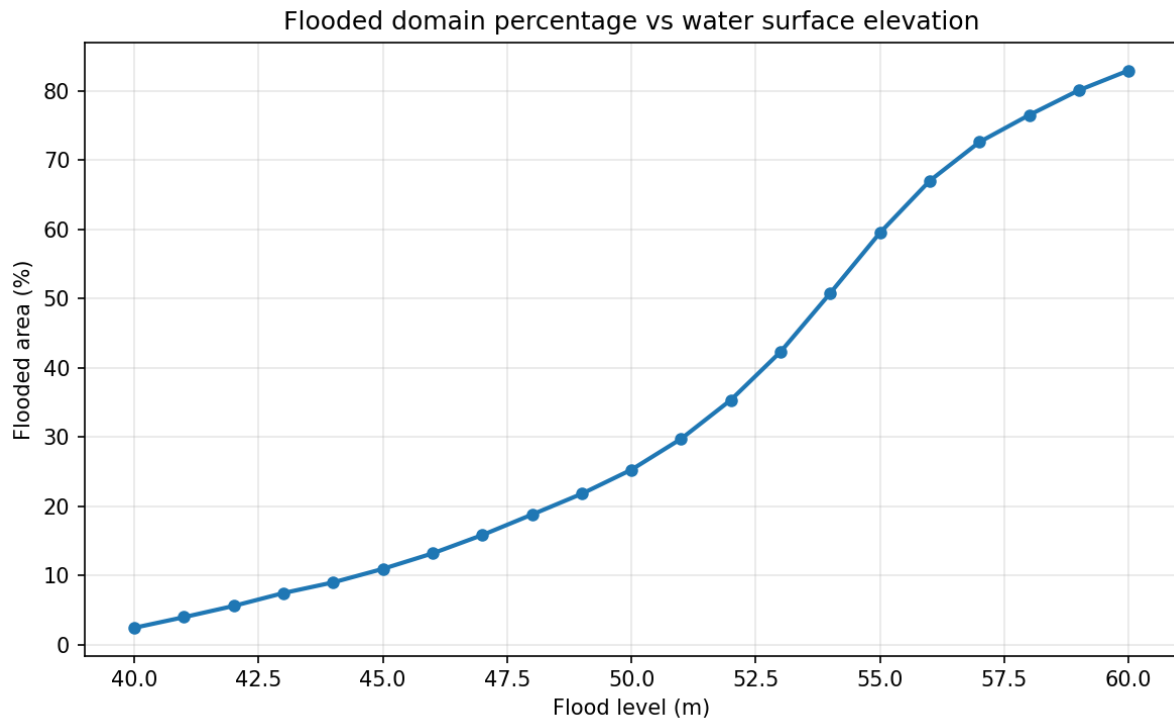


Figure 4: Flooded domain percentage vs. flood level (40–60 m, step 1 m).

At 40 m flooded area is 2.41%; at 60 m it is 82.93%. `check_monotonic_area` reports **PASS** for the 1 m step series.

5 Analysis and validation

Inclusive wet mask: Using `<=` matches a flat water surface touching cell centers at exactly stage elevation; switching to `<` would shrink area slightly—document the choice in `prompt_log.md`.

Monotonicity: For a fixed DEM and non-decreasing `flood_level`, the wet-cell count cannot shrink; PASS confirms implementation consistency.

Mean depth: Averaged over **wet cells only**, so it rises with stage both because deeper inundation and because lower-elevation cells join the wet set.

Visualization warning: `visualize_floods.py` may emit a harmless `tight_layout` warning with three subplots; the 300 DPI PNG still saves correctly (`bbox_inches=tight`).

6 Discussion

1. A single terminal screenshot is sufficient when figures carry the visual evidence (lab guide: screenshot optional).
2. Synthetic DEM with valleys and hills makes inundation statistics interpretable without downloading real GIS tiles.
3. The 50–60 m jump in flooded percent shows sensitivity to vertical datum choice in real projects.
4. No OpenCode required; generic AI prompts from the lab guide suffice for scaffolding.

7 Problems encountered

Only a matplotlib `tight_layout` user warning on the three-panel figure; output file verified on disk. No API keys or external DEM download needed.

8 Lessons learned

Document `flood_level` definition and wet/dry convention before coding. Fix the random seed early. Use one terminal screenshot plus saved PNGs for submission. If monotonicity fails, inspect mask logic and level ordering before adjusting terrain.

9 Conclusion

Week 6 Session B Lab 4 objectives were met: synthetic DEM, inundation logic, comparison maps, and monotonic rising-stage analysis with documented results in `prompt_log.md`. Submitted: `week6_session_b_lab4/`, Figure 1, and Figures 2–4.

A Core logic: `flood_inundation.py`

`flood_inundation.py` (full file)

```
#!/usr/bin/env python3
"""Flood inundation core logic -- Week 6 Session B Lab 4."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Tuple

import numpy as np

CELL_SIZE_M = 1.0
DEM_SEED = 42
DEM_PATH = "data/dem.npy"

@dataclass
class FloodStats:
    flood_level_m: float
    flooded_percent: float
    mean_depth_m: float
    flooded_area_m2: float
    wet_cell_count: int

def generate_dem(size: int = 100, seed: int = DEM_SEED) -> np.ndarray:
    """Synthetic 100x100 DEM with valleys and hills (30-80 m)."""
    rng = np.random.default_rng(seed)
    y, x = np.mgrid[0:size, 0:size]
    valley = -18.0 * np.exp(-((x - 50.0) ** 2 + (y - 50.0) ** 2) / (2.0 * 22.0**2))
    hill_nw = 14.0 * np.exp(-((x - 22.0) ** 2 + (y - 78.0) ** 2) / (2.0 * 14.0**2))
    hill_se = 12.0 * np.exp(-((x - 78.0) ** 2 + (y - 22.0) ** 2) / (2.0 * 16.0**2))
    ridge = 0.035 * (x - 0.6 * y)
    rough = rng.normal(0.0, 1.2, (size, size))
    dem = 55.0 + valley + hill_nw + hill_se + ridge + rough
    return np.clip(dem, 30.0, 80.0).astype(np.float64)
```

```

def simulate_flood(
    dem: np.ndarray, flood_level: float
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Flat water-surface inundation (absolute stage, same datum as DEM).

    Wet cells: elevation <= flood_level (inclusive at the water surface).
    Depth: max(0, flood_level - elevation) on wet cells; 0 elsewhere.
    """
    flooded_mask = dem <= flood_level
    depth = np.maximum(0.0, flood_level - dem)
    depth = np.where(flooded_mask, depth, 0.0)
    return flooded_mask, depth

def flood_statistics(
    dem: np.ndarray,
    flooded_mask: np.ndarray,
    depth: np.ndarray,
    flood_level: float,
    cell_size_m: float = CELL_SIZE_M,
) -> FloodStats:
    """Flooded area %, mean depth on wet cells, and flooded area (m^2)."""
    n_total = dem.size
    wet = int(flooded_mask.sum())
    pct = 100.0 * wet / n_total if n_total else 0.0
    if wet:
        mean_depth = float(depth[flooded_mask].mean())
    else:
        mean_depth = 0.0
    area_m2 = wet * (cell_size_m**2)
    return FloodStats(
        flood_level_m=flood_level,
        flooded_percent=pct,
        mean_depth_m=mean_depth,
        flooded_area_m2=area_m2,
        wet_cell_count=wet,
    )

def check_monotonic_area(
    levels: np.ndarray, flooded_pct: np.ndarray, tol: float = 1e-9
) -> bool:
    """Flooded percentage should not decrease as water level rises."""
    diffs = np.diff(flooded_pct)
    return bool(np.all(diffs >= -tol))

```

B DEM generator: generate_dem.py

generate_dem.py (full file)

```

#!/usr/bin/env python3
"""Exercise 1: generate and visualize synthetic DEM."""

from __future__ import annotations

import os

import matplotlib.pyplot as plt
import numpy as np

```

```

from flood_inundation import CELL_SIZE_M, DEM_PATH, DEM_SEED, generate_dem

def main() -> None:
    dem = generate_dem(size=100, seed=DEM_SEED)
    os.makedirs("data", exist_ok=True)
    np.save(DEM_PATH, dem)
    meta_path = DEM_PATH.replace(".npy", "_meta.npz")
    np.savez(
        meta_path,
        cell_size_m=CELL_SIZE_M,
        crs_note="synthetic grid, no real CRS",
        elevation_min_m=dem.min(),
        elevation_max_m=dem.max(),
        seed=DEM_SEED,
    )

    print(f"Saved DEM: {DEM_PATH} shape={dem.shape}")
    print(
        f"Elevation range: {dem.min():.2f} - {dem.max():.2f} m | "
        f"cell size: {CELL_SIZE_M} m | seed: {DEM_SEED}"
    )

    fig, ax = plt.subplots(figsize=(7, 6))
    im = ax.imshow(dem, cmap="terrain", origin="lower")
    ax.set_title("Synthetic DEM (100x100, 1 m cells)")
    ax.set_xlabel("Column index (east)")
    ax.set_ylabel("Row index (north)")
    cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    cbar.set_label("Elevation (m)")
    fig.tight_layout()
    os.makedirs("figures", exist_ok=True)
    out = "figures/dem_heatmap.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()

```

C Visualization: visualize_floods.py

visualize_floods.py (full file)

```

#!/usr/bin/env python3
"""Exercise 3: publication-style inundation maps."""

from __future__ import annotations

import os

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch

from flood_inundation import flood_statistics, simulate_flood

LEVELS = [40.0, 50.0, 60.0]

```

```

def plot_panel(dem: np.ndarray, flood_level: float, ax) -> None:
    mask, depth = simulate_flood(dem, flood_level)
    stats = flood_statistics(dem, mask, depth, flood_level)
    ax.imshow(dem, cmap="gray", origin="lower", vmin=dem.min(), vmax=dem.max())
    depth_ma = np.ma.masked_where(~mask, depth)
    im = ax.imshow(
        depth_ma,
        cmap="Blues",
        origin="lower",
        alpha=0.65,
        vmin=0,
        vmax=max(1.0, depth.max()),
    )
    ax.set_title(
        f"Flood level {flood_level:.0f} m\n"
        f"Flooded {stats.flooded_percent:.1f}% | mean depth {stats.mean_depth_m:.2f} m"
    )
    ax.set_xlabel("Column")
    ax.set_ylabel("Row")
    return im

def main() -> None:
    dem = np.load("data/dem.npy")
    fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
    last_im = None
    for ax, level in zip(axes, LEVELS):
        last_im = plot_panel(dem, level, ax)
    cbar = fig.colorbar(last_im, ax=axes, fraction=0.025, pad=0.02)
    cbar.set_label("Inundation depth (m)")
    fig.suptitle(
        "Flood inundation comparison (DEM background + depth overlay)",
        fontsize=12,
        y=1.02,
    )
    fig.tight_layout()
    os.makedirs("figures", exist_ok=True)
    out = "figures/flood_comparison_300dpi.png"
    fig.savefig(out, dpi=300, bbox_inches="tight")
    plt.close(fig)
    print(f"Saved {out}")

if __name__ == "__main__":
    main()

```

D Rising water: rising_water.py

rising_water.py (full file)

```

#!/usr/bin/env python3
"""Exercise 4: rising water level and monotonicity check."""

from __future__ import annotations

import os

import matplotlib.pyplot as plt
import numpy as np

```



```

from flood_inundation import check_monotonic_area, flood_statistics, simulate_flood

LEVEL_MIN = 40.0
LEVEL_MAX = 60.0
LEVEL_STEP = 1.0

def main() -> None:
    dem = np.load("data/dem.npy")
    levels = np.arange(LEVEL_MIN, LEVEL_MAX + LEVEL_STEP * 0.5, LEVEL_STEP)
    pcts = []
    for level in levels:
        mask, depth = simulate_flood(dem, level)
        stats = flood_statistics(dem, mask, depth, level)
        pcts.append(stats.flooded_percent)
    pcts_arr = np.array(pcts)
    monotone = check_monotonic_area(levels, pcts_arr)
    print(f"Levels {LEVEL_MIN}-{LEVEL_MAX} m step {LEVEL_STEP} m")
    print(f"Flooded % at 40 m: {pcts_arr[0]:.2f} | at 60 m: {pcts_arr[-1]:.2f}")
    print(f"Monotonicity check: {'PASS' if monotone else 'FAIL'}")
    if not monotone:
        diffs = np.diff(pcts_arr)
        bad = np.where(diffs < -1e-9)[0]
        print(f"Decreases at step index: {bad.tolist()}")

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(levels, pcts_arr, "o-", linewidth=2, markersize=5)
    ax.set_xlabel("Flood level (m)")
    ax.set_ylabel("Flooded area (%)")
    ax.set_title("Flooded domain percentage vs water surface elevation")
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    os.makedirs("figures", exist_ok=True)
    out = "figures/flooded_pct_vs_level.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    print(f"Saved {out}")

    np.savetxt(
        "figures/rising_water_data.csv",
        np.column_stack([levels, pcts_arr]),
        delimiter=",",
        header="flood_level_m,flooded_percent",
        comments="",
    )

if __name__ == "__main__":
    main()

```

E Submitted prompt log: prompt_log.md

prompt_log.md (full file)

```

# Prompt Log - Week 6 Session B Lab 4

**Student:** Mahmudul Hasan (4125999049)
**Lab:** Flood Inundation Analysis

---

```

```

## Flood_level definition used

**Absolute stage (m):** `flood_level` is the flat water surface elevation in the same
↳ datum as the DEM. A cell is wet when `elevation <= flood_level`. Depth is `max(0,
↳ flood_level - elevation)` on wet cells and `0` elsewhere.

---

## Cell size assumption

**1 m** square cells on a 100x100 synthetic grid (10,000 cells, 10,000 m2 domain). Area
↳ statistics use cell count x 1 m2 unless noted.

---

## Exercise 2 results (flood levels)

| Flood level (m) | Flooded % | Mean depth on wet (m) |
|-----|-----|-----|
| 40 | 2.41 | 1.183 |
| 50 | 25.25 | 4.863 |
| 60 | 82.93 | 8.329 |

---

## Monotonicity check

**Status:** PASS

**Levels swept:** 40 to 60 m, step 1 m

**Figure:** `figures/flooded_pct_vs_level.png`

---

## Figure files

- `figures/dem_heatmap.png` -- Exercise 1 DEM
- `figures/flood_comparison_300dpi.png` -- Exercise 3 side-by-side (40, 50, 60 m)
- `figures/flooded_pct_vs_level.png` -- Exercise 4

---

## Lessons learned

Define wet/dry and depth conventions in `prompt_log.md` before plotting. A fixed random
↳ seed keeps DEM and submission figures reproducible. Rising-stage curves should be
↳ monotone non-decreasing; if not, check mask logic (`<=` vs `<`). This lab does not
↳ require OpenCode; local Python and the lab guide AI prompts are enough.

```